Section 1: Giới thiệu kiến trúc RAG

Hệ thống RAG (Retrieval-Augmented Generation) là một kỹ thuật tiên tiến trong xử lý ngôn ngữ tự nhiên, kết hợp giữa khả năng truy xuất thông tin (retrieval) và mô hình ngôn ngữ lớn (LLM). Hệ thống RAG bao gồm 3 thành phần cốt lõi:

1. Document Retriever: Thành phần phân tích và trích xuất tài liệu nguồn.

2. Vector Store: Cơ sở dữ liệu vector dùng để lưu trữ và lập chỉ mục các embedding của từng đoạn văn bản.

3. Large Language Model (LLM): Mô hình sinh văn bản (như Gemini 1.5 Flash) tiếp nhận ngữ cảnh và tạo câu trả lời tự nhiên.

Mục tiêu chính của RAG là giải quyết vấn đề ảo giác (hallucination) của AI và cung cấp cho AI khả năng truy cập tri thức cập nhật từ các tài liệu riêng tư của người dùng mà không cần huấn luyện lại mô hình.

Section 2: Vai trò của Embedding Model

Embedding Model (như Google Gemini text-embedding-004) đóng vai trò biến đổi văn bản tự nhiên thành các vector số học trong không gian nhiều chiều (ví dụ 768 chiều).

Mỗi từ, câu hoặc đoạn văn bản sau khi qua mô hình embedding sẽ trở thành một tọa độ vector đặc trưng cho ngữ nghĩa của nó.

Khi tìm kiếm thông tin, hệ thống sử dụng thuật toán tính khoảng cách như Cosine Similarity (độ tương đồng Cosine) để so sánh vector câu hỏi của người dùng với các vector tài liệu đã lưu, từ đó nhanh chóng tìm ra các đoạn có ý nghĩa gần gũi nhất, kể cả khi không trùng khớp chính xác từng từ khóa.

Section 3: Vector Store trên bộ nhớ RAM

Trong bài tập này, toàn bộ vector embedding được lưu trữ trực tiếp trên bộ nhớ RAM (In-Memory Vector Store) thay vì lưu xuống các cơ sở dữ liệu chuyên dụng bên ngoài như Qdrant, ChromaDB hay pgvector.

Ưu điểm của giải pháp lưu trữ RAM:

- Tốc độ đọc ghi và tính toán tương đồng cực kỳ nhanh, độ trễ thường dưới 5 mili-giây.

- Đơn giản hóa kiến trúc triển khai, không yêu cầu cài đặt phần mềm Docker hay thiết lập database phức tạp.

Nhược điểm của giải pháp RAM:

- Dữ liệu vector chỉ tồn tại tạm thời trong suốt vòng đời của tiến trình ứng dụng; khi server restart thì dữ liệu trong RAM sẽ bị xóa sạch và cần nạp lại từ đầu.

- Giới hạn dung lượng phụ thuộc vào dung lượng bộ nhớ vật lý của máy chủ.

Section 4: Quy trình truy vấn (Query Pipeline)

Quy trình truy vấn trong hệ thống RAG diễn ra theo 4 bước tuần tự như sau:

Bước 1: Người dùng gửi câu hỏi qua giao diện chat Front-end.

Bước 2: Back-end nhận câu hỏi và đưa vào Embedding Model để tạo vector truy vấn.

Bước 3: Thực hiện tìm kiếm vector tương đồng (Vector Similarity Search) trong kho lưu trữ RAM, trích xuất Top-K đoạn tài liệu có điểm tương đồng cao nhất kèm theo metadata (Tên file, Section, Số trang).

Bước 4: Ghép các đoạn trích dẫn vào prompt gửi tới Gemini LLM cùng với chỉ dẫn nghiêm ngặt yêu cầu trả lời dựa trên trích dẫn và nêu rõ nguồn gốc section. Cuối cùng, phản hồi kết quả và danh sách trích dẫn về Front-end để hiển thị cho người dùng.

Section 5: Cơ chế trích dẫn Section và Nguồn tài liệu

Để đảm bảo tính minh bạch và cho phép người dùng kiểm chứng kết quả trả về, hệ thống triển khai cơ chế gán nhãn metadata phân cấp:

- Mỗi khi đọc tài liệu (PDF hoặc DOCX), bộ phân tích văn bản sẽ quét qua từng trang và nhận diện các tiêu đề Section (ví dụ: Section 1, Chương 2, Điều 3...).

- Mỗi đoạn văn (chunk) được gắn chặt với thông tin: tên file gốc, số trang thực tế và tên section gần nhất.

- Khi gửi prompt cho mô hình Gemini, hệ thống cung cấp rõ ràng thông tin nguồn của từng đoạn trích.

- Mô hình Gemini được huấn luyện chỉ dẫn (prompt instruction) để trích dẫn cụ thể tên Section trong phần trả lời, giúp người dùng đối chiếu tức thì nội dung trong tài liệu gốc.